PROJECT PHASE - II

Indexed Tag-Based Knowledge Retrieval System

An Project Phase - II Report Submitted
In partial fulfillment of the requirements for the award of the degree
of

Bachelor of Technology
in
Computer Science and Engineering
by

SK. AFROZ	**22N31A05M3**
S. AKSHARA	**22N31A05N7**
T. SIVA RAMA KRISHNA	**22N31A05P7**

Under the esteemed guidance of
DR. M. SAMBASIVUDU
Associate Professor

Department of Computer Science and Engineering

Malla Reddy College of Engineering & Technology
(Autonomous Institution- UGC, Govt. of India)
(Affiliated to JNTUH, Hyderabad, Approved by AICTE, NBA &NAAC with 'A' Grade)
Maisammaguda, Kompally, Dhulapally, Secunderabad – 500100
website: www.mrcet.ac.in

2022-2026

Malla Reddy College of Engineering & Technology

(Autonomous Institution- UGC, Govt. of India)
(Affiliated to JNTUH, Hyderabad, Approved by AICTE, NBA &NAAC with 'A' Grade)
Maisammaguda, Kompally, Dhulapally, Secunderabad – 500100
website: www.mrcet.ac.in

CERTIFICATE

This is to certify that this is the bonafide record of the project entitled "**Indexed Tag-Based Knowledge Retrieval System**", submitted by **Sk. Afroz(22N31A05M3), S. Akshara (22N31A05N7) and T. Siva Rama Krishna (22N31A05P7)** of B. Tech in the partial fulfilment of the requirements for the degree of Bachelor of Technology in Computer Science and Engineering, Department of CSE during the year 2025-2026. The results embodied in this project report have not been submitted to any other university or institute for the award of any degree or diploma.

Internal Guide	Project Coordinator	Head of the Department
Dr. M. Sambasivudu	**Dr. M. Sambasivudu**	**Dr. G. Ravi**
Associate Professor	**Associate Professor**	**Professor**

External Examiner

DECLARATION

We hereby declare that the application titled "Indexed Tag-Based Knowledge Retrieval System" submitted to Malla Reddy College of Engineering and Technology (UGC Autonomous), affiliated to Jawaharlal Nehru Technological University Hyderabad (JNTUH) for the award of the degree of Bachelor of Technology in Computer Science and Engineering is a result of original research carried-out in this thesis. It is further declared that the project report or any part thereof has not been previously submitted to any University or Institute for the award of degree or diploma.

SK. AFROZ	**22N31A05M3**
S. AKSHARA	**22N31A05N7**
T. SIVA RAMA KRISHNA	**22N31A05P7**

ACKNOWLEDGEMENT

We feel honored to place our warm salutation to our college Malla Reddy College of Engineering and Technology (UGC-Autonomous) for giving us an opportunity to do this Project as part of our B. Tech Program. We are ever grateful to our **Principal Dr.V.Murali Mohan** who enabled us to have experience in engineering and gain profound technical knowledge.

We express our heartiest thanks to our **HOD, Dr. G.Ravi** for encouraging us in every aspect of our course and helping us realize our full potential.

We would like to thank our **Project Guide** and **Project Coordinator Dr.M.Sambasivudu** for his continuous monitoring andunflinching co-operation throughout project work.

We would like to thank our **Class Incharge Dr.M.Sambasivudu** who in spite of being busy with his academic duties took time to guide and keep us on the correct path.

We would also like to thank all the faculty members and supporting staff of the Department of CSE and all other departments who have been helpful directly or indirectlyin making our project a success.

We are extremely grateful to our parents for their blessings and prayers for the completionof our project that gave us strength to do our project.

With regards and gratitude

SK. AFROZ	**22N31A05M3**
S. AKSHARA	**22N31A05N7**
T. SIVA RAMA KRISHNA	**22N31A05P7**

ABSTRACT

Managing personal knowledge in the form of notes, diagrams, and reference links often becomes inefficient as the volume of information grows. Existing tools either rely on simple keyword searches or introduce unnecessary complexity that reduces usability. This project presents a low-latency knowledge storage and retrieval system that organizes content using indexed tags and supports intuitive querying. The system allows users to store different types of content locally and retrieve them instantly through tag-based searches. An indexed backend ensures fast query execution, while optional caching improves response time for frequently accessed data. The platform follows a modular and local-first architecture, making it suitable for self-hosted usage without dependency on cloud services. The project emphasizes clean system design, performance optimization, and practical usefulness over advanced machine learning or heavy research components.

TABLE OF CONTENTS

S.NO	TITLE	PG.NO
1	**INTRODUCTION**	**01**
	1.1 PURPOSE AND OBJECTIVES	02
	1.2 EXISTING AND PROPOSED SYSTEM	05
	1.3 SCOPE OF PROJECT	08
2	**LITERATURE SURVEY**	**10**
3	**SYSTEM ANALYSIS**	**11**
	3.1 HARDWARE AND SOFTWARE REQUIREMENTS	**11**
	3.2 SOFTWARE REQUIREMENTS SPECIFICATION	12
4	**SYSTEM DESIGN**	**15**
	4.1 DESCRIPTION	15
	4.2 ARCHITECTURE	19
	4.3 UML DIAGRAMS	20
5	**METHODOLOGY**	**28**
	5.1 MODULE DESCRIPTION	28
	5.2 PROCESS / ALGORITHM	29
6	**IMPLEMENTATION**	**30**
	6.1 SAMPLE CODE	30
	6.2 OUTPUT SCREENS	44
7	**CONCLUSION & FUTURE SCOPE**	49
8	**BIBLIOGRAPHY**	**50**

LIST OF FIGURES

FIGURE.NO	NAME	PG.NO
4.2	ARCHITECTURE	19
4.3.1	USE CASE DIAGRAM	20
4.3.2	SEQUENCE DIAGRAM	21
4.3.3	CLASS DIAGRAM	22
4.3.4	ACTIVITY DIAGRAM	23
4.3.5	COLLABORATION DIAGRAM	24
4.3.6	STATE CHART	25
4.3.7	COMPONENT DIAGRAM	26
4.3.8	DEPLOYMENT DIAGRAM	27

LIST OF OUTPUTS

FIGURE.NO	NAME	PG.NO
6.2.1	Code snippet in IDE	44
6.2.2	Main landing page	45
6.2.3	Content Display page	46
6.2.4	Content creation page	47
6.2.5	RAG chatbot interface	48

1.INTRODUCTION

The rapid growth of digital information has made personal knowledge management an important challenge for students and professionals. Individuals frequently store information in the form of notes, diagrams, documents, and web links. However, traditional methods such as folder-based organization or simple keyword search often become inefficient as the volume of data increases. These approaches lack structure and result in slower retrieval, making it difficult to access relevant information when needed.

In many existing systems, users depend on remembering file names, locations, or manually browsing through folders, which leads to reduced productivity. Even modern tools that provide search functionality often rely on full-text scanning or basic matching techniques, which may not scale efficiently for large datasets. As a result, there is a need for a system that provides structured organization along with fast and reliable retrieval of information.

This project presents an **Indexed Knowledge Storage and Retrieval System**, designed to improve the efficiency of storing and accessing personal knowledge. The system uses a tag-based indexing mechanism to organize data and enable fast retrieval without performing full database scans. By associating each item with meaningful tags, the system allows users to quickly locate relevant information using simple queries.

The system supports multiple content types, including text notes, images, and links, providing flexibility in managing different forms of knowledge. It is designed using a modular client–server architecture, where a backend handles data processing and storage while multiple clients interact with the system. The use of indexing and caching techniques ensures that frequently accessed data can be retrieved with minimal delay.

Additionally, the system follows a local-first approach, allowing users to maintain control over their data without relying entirely on cloud services. This improves reliability, privacy, and accessibility. The design also supports future enhancements such as cloud integration and advanced query mechanisms.

Overall, the project focuses on delivering a simple yet efficient solution for personal knowledge management by combining structured data organization, fast retrieval, and practical system design principles.

1.1PURPOSE AND OBJECTIVES

PURPOSE:

The purpose of this project is to design and develop an efficient system for storing, organizing, and retrieving personal knowledge in a structured and reliable manner. With the rapid growth of digital information, individuals frequently deal with large volumes of data such as notes, diagrams, documents, and web links. Managing this information effectively has become a significant challenge, especially when traditional systems fail to provide quick and meaningful access to stored content.

In conventional systems, users rely heavily on folder structures, file names, or manual browsing to locate information. As the amount of data increases, these methods become inefficient and time-consuming. Users often struggle to recall where specific information is stored, resulting in delays and reduced productivity. This highlights the need for a more structured and intelligent approach to knowledge management.

The primary purpose of this project is to overcome these limitations by introducing a **tag-based indexing system**. In this approach, each knowledge item is associated with one or more tags that describe its content. These tags are then indexed, allowing the system to directly retrieve relevant data without scanning the entire dataset. This significantly improves search efficiency and reduces retrieval time.

Another important purpose of the system is to support **multiple content types** such as text notes, images, and web links within a unified platform. Users often store information in different formats, and managing them separately can be inconvenient. By integrating all content types into a single system, the project provides a more organized and user-friendly solution.

The project also focuses on implementing a **local-first architecture**, where data is stored and managed locally rather than relying entirely on cloud services. This ensures better data privacy, control, and reliability. Users are not dependent on internet connectivity for accessing their data, which makes the system more robust and accessible.

In addition, the system aims to achieve **high performance** through the use of indexing and caching mechanisms. By minimizing unnecessary database operations and optimizing query processing, the system ensures low latency and quick response times. This is

particularly important for applications where users expect instant access to their information.

Finally, the purpose of this project is to create a **modular and scalable system design** that can be extended in the future. The architecture allows integration of additional features such as cloud synchronization, mobile access, and advanced query processing without requiring major changes to the core system.

OBJECTIVES

The main objective of this project is to develop a system that enables efficient storage, organization, and retrieval of personal knowledge. The system is designed to improve accessibility, reduce search time, and provide a structured way of managing different types of information.

Primary Objectives

- To design a system that can store multiple types of data including notes, images, and links in a unified format
- To implement a tag-based indexing mechanism for efficient data organization
- To enable fast retrieval of stored information using indexed lookup instead of full database scanning
- To reduce the time required to locate specific information within large datasets

System Design Objectives

- To develop a modular backend architecture that separates different system components
- To ensure that the system is easy to maintain, update, and extend
- To design RESTful APIs for communication between clients and backend
- To support multiple client interfaces such as web applications and bot-based systems

Performance Objectives

- To improve system performance using indexing techniques
- To integrate caching mechanisms for faster access to frequently used data
- To ensure low response time for search queries
- To optimize database operations for better efficiency

Usability Objectives

- To provide a simple and intuitive interface for users

- To minimize the learning curve for using the system
- To ensure that users can easily store and retrieve information
- To allow flexible tagging for better organization

Reliability and Security Objectives

- To ensure data consistency and integrity during storage and retrieval
- To provide reliable data storage with backup support
- To maintain user control over data through a self-hosted system
- To reduce dependency on external cloud services

Scalability Objectives

- To design the system in such a way that it can handle increasing data volume
- To ensure that system performance does not degrade significantly as data grows
- To support future enhancements such as cloud integration and multi-user access

Future-Oriented Objectives

- To create a foundation for advanced query mechanisms
- To enable integration with mobile platforms
- To support additional features such as analytics and usage tracking
- To prepare the system for expansion into larger-scale applications

1.2EXISTING AND PROPOSED SYSTEM

EXISTING

- In the current digital environment, knowledge management is typically handled using basic tools such as file systems, note-taking applications, and cloud-based platforms. These systems allow users to store data in the form of documents, images, and links. However, most of these approaches rely on either manual organization or simple keyword-based search mechanisms, which become inefficient as the volume of data increases.
- In traditional file-based systems, users organize their data into folders and subfolders. While this method is simple, it lacks flexibility. A single file can belong to only one folder, even though it may be relevant to multiple topics. This forces users to either duplicate files or remember specific folder paths, making retrieval difficult. As the data grows, navigating through multiple levels of directories becomes time-consuming and error-prone.
- Modern note-taking applications provide better interfaces and allow users to store structured content. However, many of these systems still rely on full-text search or basic filtering techniques. When a user performs a search, the system scans through large amounts of data to find matches. This process becomes slower as the dataset increases, resulting in higher response times. Additionally, keyword-based search may not always return accurate results if the exact terms are not used.
- Cloud-based systems offer accessibility and synchronization across devices, but they introduce other limitations. Users must depend on internet connectivity to access their data, which can be inconvenient in offline situations. There are also concerns related to data privacy, security, and long-term dependency on external platforms. Some services may require subscriptions, adding financial cost to the user.
- Another limitation of existing systems is the lack of efficient indexing for personal knowledge. While large-scale search engines use advanced indexing techniques, many personal tools do not implement optimized indexing mechanisms. This results in inefficient data retrieval and reduced system performance when handling larger datasets.

- Furthermore, most existing systems are not designed with modularity and extensibility in mind. Integrating new features or customizing the system for specific needs can be difficult. Users are often restricted by the limitations of the platform and cannot modify the internal functionality.

Limitations of Existing Systems

- Dependence on manual organization using folders and file names
- Inefficient retrieval due to lack of proper indexing
- Slow performance when handling large datasets
- Reliance on full-text or keyword-based search
- Dependency on internet connectivity in cloud-based systems
- Data privacy and security concerns
- Limited flexibility in organizing data across multiple categories
- Lack of modular and customizable system design

PROPOSED

- To overcome the limitations of existing systems, the proposed solution introduces an **Indexed Knowledge Storage and Retrieval System** that focuses on efficient data organization and fast retrieval. The system is designed using a tag-based indexing approach, where each knowledge item is associated with one or more tags. These tags are indexed, allowing the system to directly locate relevant data without scanning the entire dataset.
- Unlike traditional folder-based systems, the proposed system provides flexible organization. A single item can have multiple tags, making it easier to categorize information across different topics. This eliminates the need for duplication and improves data accessibility. Users can retrieve information using simple tag queries or structured search inputs, making the system both powerful and easy to use.
- The system supports multiple types of content, including notes, images, and links. All data is stored in a unified structure, allowing users to manage different forms of knowledge within a single platform. This improves consistency and reduces fragmentation of information.
- A key feature of the proposed system is the use of **indexing mechanisms** to enable fast retrieval. Instead of performing full database scans, the system uses

precomputed indexes that map tags to items. This significantly reduces search time and ensures low latency, even as the dataset grows.

- To further enhance performance, a caching layer is implemented. Frequently accessed queries and results are stored in memory, allowing the system to return results quickly without repeatedly accessing the database. This improves overall efficiency and reduces system load.
- The system is designed with a **local-first architecture**, ensuring that data is stored and managed locally. This provides better control over data, improves privacy, and allows offline access. At the same time, the architecture is flexible enough to support future integration with cloud services if required.
- Another important aspect of the proposed system is its **modular design**. The system is divided into independent components such as data management, indexing, search, and caching. This makes it easier to maintain, update, and extend the system with new features.

Advantages of Proposed System

- Fast retrieval using indexed tag-based search
- Flexible data organization with multiple tags per item
- Reduced search time and improved performance
- Support for multiple content types (notes, images, links)
- Local-first architecture ensuring data ownership and privacy
- Integration of caching for improved response time
- Modular and scalable system design
- Reduced dependency on external cloud platforms
- Easy to extend with additional features in the future

1.3SCOPE OF THE PROJECT

- The scope of the **Indexed Knowledge Storage and Retrieval System** defines the boundaries, functionalities, and limitations of the system. It outlines what the system is designed to achieve and specifies the areas that are included and excluded from the project. The primary focus of this system is to provide an efficient solution for storing, organizing, and retrieving personal knowledge in a structured and reliable manner.
- The system is intended for individual users who need to manage various forms of digital information such as notes, images, and web links. It provides a unified platform where all types of content can be stored and accessed through a common interface. By using a tag-based indexing mechanism, the system ensures that users can quickly retrieve relevant information without manually searching through large datasets.
- The scope of the project includes the development of a modular backend system that handles data storage, indexing, and retrieval. The system supports core operations such as creating, updating, deleting, and retrieving knowledge items. It also includes tag management features, allowing users to assign multiple tags to each item for flexible organization.
- The system is designed to provide fast and efficient retrieval using indexed lookup techniques. Instead of scanning the entire database during a search, the system uses precomputed indexes to directly locate relevant items. This significantly improves performance and reduces response time, especially as the volume of data increases.
- Another important aspect within the scope is the implementation of a caching mechanism. Frequently accessed data is stored in memory to reduce repeated database queries and improve system efficiency. This ensures that commonly used information can be retrieved almost instantly.
- The project also includes support for multiple client interfaces. Users can interact with the system through a web interface for detailed management and through bot-based platforms such as Telegram or Discord for quick access. This multi-interface design improves accessibility and usability.The system follows a **local-first approach**, meaning that all data is stored and managed locally. This ensures data privacy, security, and independence from cloud-based services. Users retain full

control over their data, and the system can function without continuous internet connectivity, except for optional bot integration.

In-Scope Features

The following functionalities are included within the scope of the project:

- Storage and management of knowledge items (notes, images, and links)
- Tag-based indexing for efficient organization of data
- Fast retrieval using indexed search mechanisms
- Support for simple and structured query-based searching
- CRUD operations (Create, Read, Update, Delete)
- Implementation of caching to improve performance
- Modular backend architecture
- Multi-client access through web and bot interfaces
- Local file storage and database management
- Containerized deployment using Docker

Out-of-Scope Features

The following aspects are not included in the current scope of the project:

- Large-scale distributed or enterprise-level deployment
- Multi-user collaboration and role-based access control
- Advanced artificial intelligence or machine learning features
- Real-time synchronization across multiple devices
- Heavy cloud-based infrastructure or SaaS deployment
- Complex multimedia editing or processing

Future Scope

The system is designed in a way that allows future enhancements and extensions, including:

- Integration with cloud storage for remote access
- Development of mobile applications for better accessibility
- Implementation of advanced query language and filtering
- Addition of user authentication and multi-user support
- Data analytics and usage tracking features
- Automated backup and recovery mechanisms

2.LITERATURE SURVEY

- Personal knowledge management has been an active area of development in both academia and industry, focusing on efficient storage, organization, and retrieval of user-generated information. Traditional approaches rely on file systems and keyword-based search techniques, which often become inefficient as data volume increases. Studies in information retrieval emphasize the importance of indexing techniques, such as inverted indexing, to improve search performance and reduce retrieval time. These methods form the foundation of modern search systems and highlight the need for structured data organization.
- Several existing tools and platforms have addressed knowledge management from different perspectives. Applications like Notion and Evernote provide user-friendly interfaces and cloud-based storage, enabling users to organize notes and multimedia content. Similarly, Obsidian focuses on local-first knowledge management with linked notes and graph-based visualization. While these tools offer flexibility and ease of use, they often rely on full-text search or basic indexing and may introduce dependencies on cloud infrastructure or proprietary ecosystems.
- Research in database systems highlights the advantages of relational databases such as PostgreSQL for structured data management and efficient querying. Additionally, caching systems like Redis are widely used to improve application performance by reducing database load and latency. Combining indexing techniques with caching mechanisms has been shown to significantly enhance system responsiveness, especially in applications requiring frequent data retrieval.
- The proposed system builds upon these concepts by integrating tag-based indexing with a modular backend architecture. Unlike traditional systems that depend on full-text search, this approach emphasizes direct indexed lookup for faster performance. Furthermore, the system adopts a local-first design, ensuring data ownership and minimizing external dependencies. By combining established database principles with practical system design, the project provides a simplified yet efficient solution for personal knowledge management.

3.SYSTEM ANALYSIS

3.1HARDWARE AND SOFTWARE REQUIREMENTS

HARDWARE REQUIREMENTS:

The system is designed to run efficiently on standard computing devices without requiring specialized hardware. The following configuration is sufficient for development and deployment:

- Processor: Intel i3 / AMD equivalent or higher
- RAM: Minimum 8 GB (Recommended for smooth performance)
- Storage: At least 10 GB of free disk space for application, database, and file storage
- Network: Internet connection (required only for bot integration and optional updates)
- System Type: 64-bit architecture

SOFTWARE REQUIREMENTS:

The system is developed using modern backend technologies and open-source tools to ensure flexibility and scalability.

Operating System

- Windows / Linux / macOS

Backend Technologies

- FastAPI – for building REST APIs
- PostgreSQL – for structured data storage
- Redis – for caching and performance optimization

Development Tools

- Python 3.x
- Docker & Docker Compose – for containerized deployment
- Git – for version control

Client Interfaces

- Web Browser (for web interface)
- Telegram / Discord – for bot-based interaction

3.2 SOFTWARE REQUIREMENTS SPECIFICATION

The Software Requirements Specification (SRS) defines the complete functional and non-functional requirements of the Indexed Knowledge Storage and Retrieval System. It provides a detailed description of system behavior, features, and constraints, serving as a reference for design, development, and testing.

1. System Overview

The system is designed to store, manage, and retrieve personal knowledge items such as notes, images, and links using indexed tags. It follows a client–server architecture with a modular backend and supports multiple interfaces such as a web client and bot-based access.

2. Functional Requirements

2.1 Data Management

- The system shall allow users to create and store knowledge items
- The system shall support multiple content types (text, images, links)
- The system shall allow updating and deletion of stored items

2.2 Tag Management

- The system shall allow assigning one or more tags to each item
- The system shall maintain a mapping between tags and items
- The system shall support modification and removal of tags

2.3 Search and Retrieval

- The system shall retrieve items using tag-based indexing
- The system shall support multi-tag and filtered search queries
- The system shall return results with low latency

2.4 User Interaction

- The system shall provide access through a web interface
- The system shall support bot-based interaction (Telegram/Discord)
- The system shall provide a simple and intuitive interface

2.5 System Operations

- The system shall support CRUD operations via REST APIs
- The system shall handle multiple client requests efficiently
- The system shall maintain data consistency

3. Non-Functional Requirements

3.1 Performance

- The system shall provide fast retrieval using indexing
- The system shall reduce response time using caching mechanisms

3.2 Reliability

- The system shall ensure data integrity and persistence
- The system shall support backup and recovery mechanisms

3.3 Scalability

- The system shall handle increasing data without major performance loss

3.4 Usability

- The system shall be easy to use with minimal learning effort

3.5 Maintainability

- The system shall follow a modular and extensible design

3.6 Portability

- The system shall be deployable across multiple operating systems using containerization

4. System Constraints

- The system is limited to personal or small-scale usage
- Requires basic technical knowledge for setup and deployment
- Depends on local system resources for performance
- Does not include advanced AI-based features

5. Assumptions

- Users have basic familiarity with storing and retrieving digital content
- The system will be used primarily for personal knowledge management
- The deployment environment supports required technologies (Python, Docker, database systems)

4.SYSTEM DESIGN

4.1 DESCRIPTION

1. Purpose:

System design defines the overall structure, components, and interactions of the **Indexed Knowledge Storage and Retrieval System**. The design focuses on modularity, performance, and simplicity, ensuring that the system is easy to develop, maintain, and scale.

1. Design Overview

The system follows a **client–server architecture**, where multiple clients interact with a centralized backend. The backend processes requests, manages data storage, performs indexing, and returns results efficiently.

The design is divided into the following layers:

- **Presentation Layer (Client Interfaces)**
- **Application Layer (Backend API)**
- **Data Layer (Database and Storage)**
- **Support Layer (Indexing and Caching)**

2. Architectural Design

The architecture consists of interconnected components that handle specific responsibilities:

2.1 Client Layer

- Web interface for managing and viewing knowledge items
- Bot interface (Telegram/Discord) for quick retrieval
- Sends user requests to the backend via API

2.2 Backend Layer

- Developed using FastAPI
- Handles request processing and business logic
- Implements CRUD operations and query handling
- Communicates with database, cache, and storage

2.3 Data Layer

- PostgreSQL stores structured metadata (items, tags, relationships)
- Local file system stores images and files
- Ensures data persistence and integrity

2.4 Caching Layer

- Redis stores frequently accessed queries and results
- Reduces database load and improves response time

2.5 Indexing Layer

- Maintains tag-to-item mappings
- Enables fast lookup without scanning entire data
- Updated dynamically when data is added or modified

3. Data Flow Design

The system processes data in the following sequence:

1. User sends a request through web or bot interface
2. Backend API receives and validates the request
3. Query is checked in cache (Redis)
    - If found → return cached result

    - If not found → proceed to database

4. Backend uses index to locate relevant items in database
5. Results are retrieved and returned to the client
6. Frequently accessed results are stored in cache

4. Database Design (Overview)

The system uses a relational schema consisting of:

- **Items Table** → stores notes, links, and file references
- **Tags Table** → stores tag information
- **Item-Tag Mapping Table** → maintains many-to-many relationships

Indexes are created on tag fields to ensure fast retrieval.

5. Modular Design

The system is divided into independent modules:

- Data Management Module
- Tag Management Module
- Search and Retrieval Module
- Caching Module
- API Layer

Each module operates independently, improving maintainability and scalability.

6. Deployment Design

- The system is containerized using Docker
- Services include backend, database, and cache

- Can be deployed locally with a single command
- Ensures consistent environment across systems

4.2 ARCHITECTURE

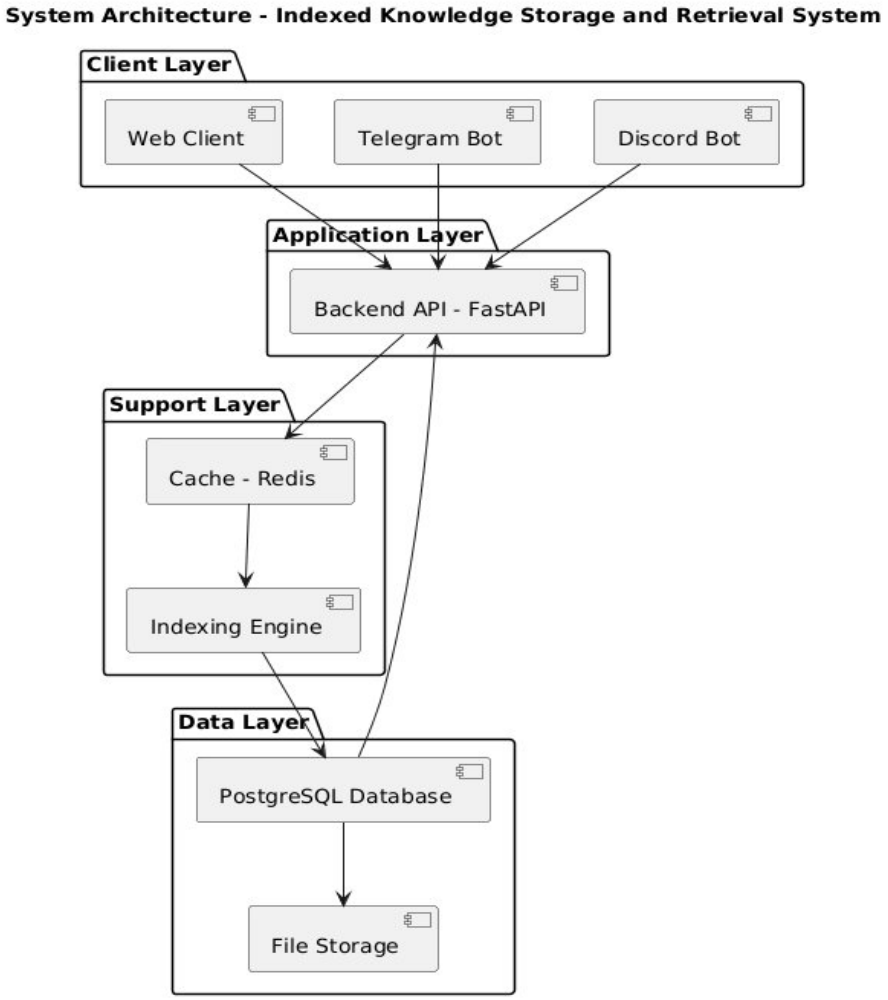


Fig : 4.2 Architecture

The system architecture is designed using a layered approach that ensures efficient organization and processing of data. It consists of multiple layers, including the client layer, application layer, support layer, and data layer. The client layer provides user interfaces such as web applications and bots, through which users interact with the system. The application layer, which includes the backend API, processes user requests and manages system logic. The support layer includes caching and indexing mechanisms that enhance system performance. The cache stores frequently accessed data, reducing the need for repeated database queries, while the indexing system enables fast retrieval of data based on tags. The data layer consists of the database and file storage, which ensure persistent and organized storage of information. The overall architecture ensures that user requests are processed efficiently, data is retrieved quickly, and the system remains scalable, reliable, and easy to maintain.

4.3 UML DIAGRAMS

4.3.1 USE CASE DIAGRAM:

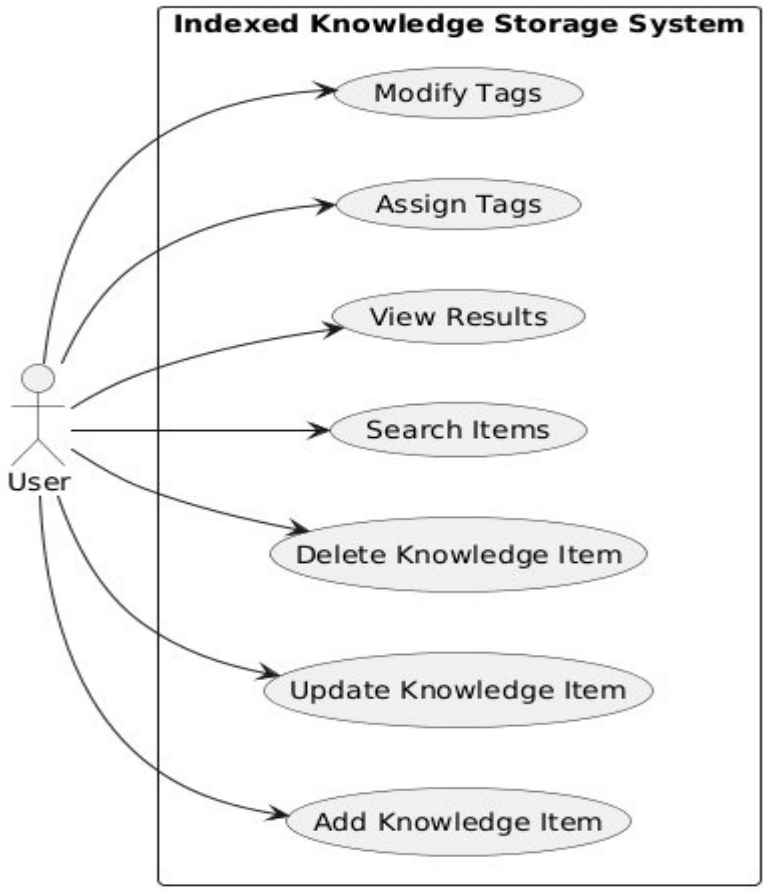


Fig : 4.3.1 Use case diagram

The Use Case Diagram represents the interaction between the user and the Indexed Knowledge Storage and Retrieval System. It identifies the various functionalities that the system provides, such as adding new knowledge items, updating existing data, deleting unwanted entries, and searching for stored information using tags. It also includes additional operations like viewing retrieved results and managing tags assigned to different items. This diagram helps in clearly defining the system boundary by showing what actions are handled within the system and what actions are initiated by the user.

The diagram is useful for understanding the system from a functional perspective, as it highlights all possible operations that the user can perform. It ensures that the system requirements are properly captured and that no essential functionality is missing during development. By providing a high-level view of user interactions, the Use Case Diagram acts as a foundation for further design and development phases.

4.2.2 SEQUENCE DIAGRAM:

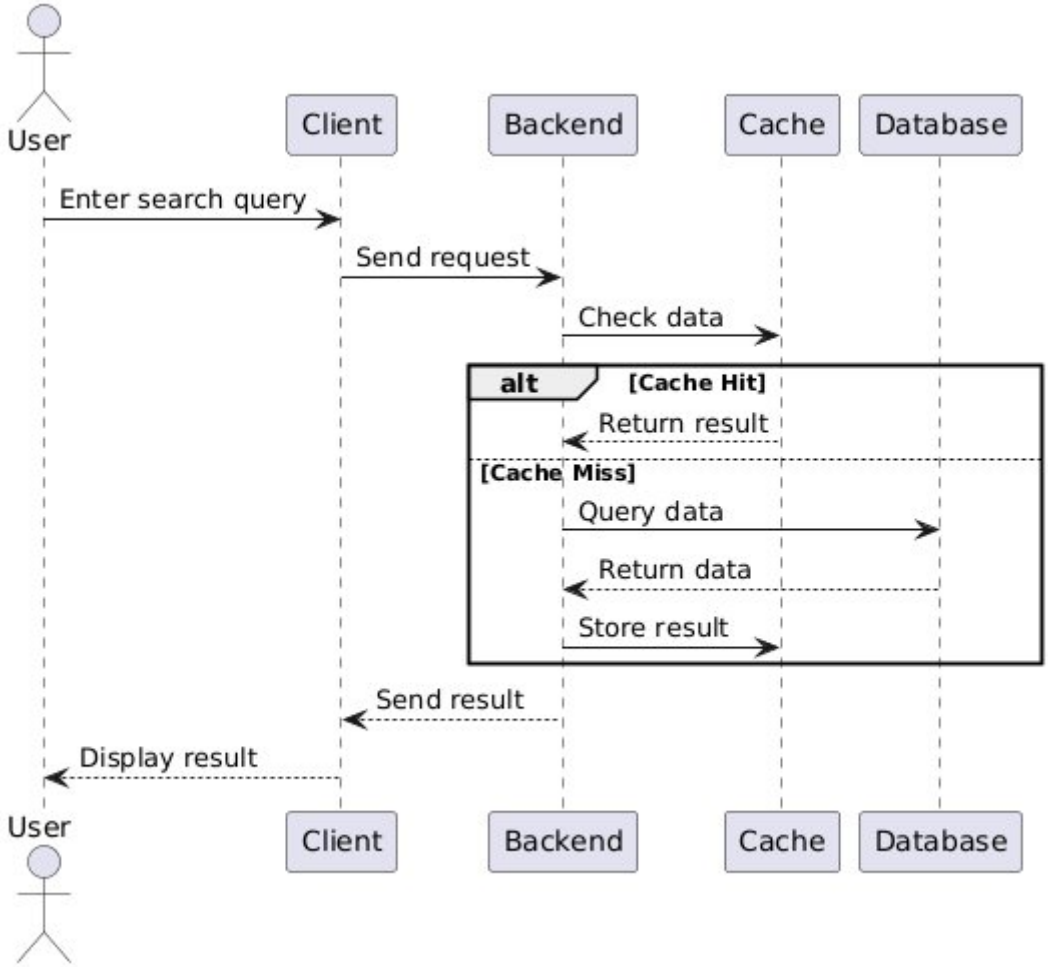


Fig : 4.3.2 Sequence diagram

The Sequence Diagram illustrates the dynamic behavior of the system by showing how different components interact over time to complete a specific task. It represents the sequence of messages exchanged between the user, client interface, backend server, cache, and database. For example, during a search operation, the diagram shows how the request is sent from the user to the backend, how the cache is checked, and how the database is queried if the data is not found in cache.

This diagram is particularly useful for understanding the flow of operations and the order in which processes occur. It helps in identifying dependencies between components and ensures that the system behaves correctly during execution. By visualizing the interaction step by step, the Sequence Diagram aids in debugging and improving system efficiency.

4.3.3 CLASS DIAGRAM:

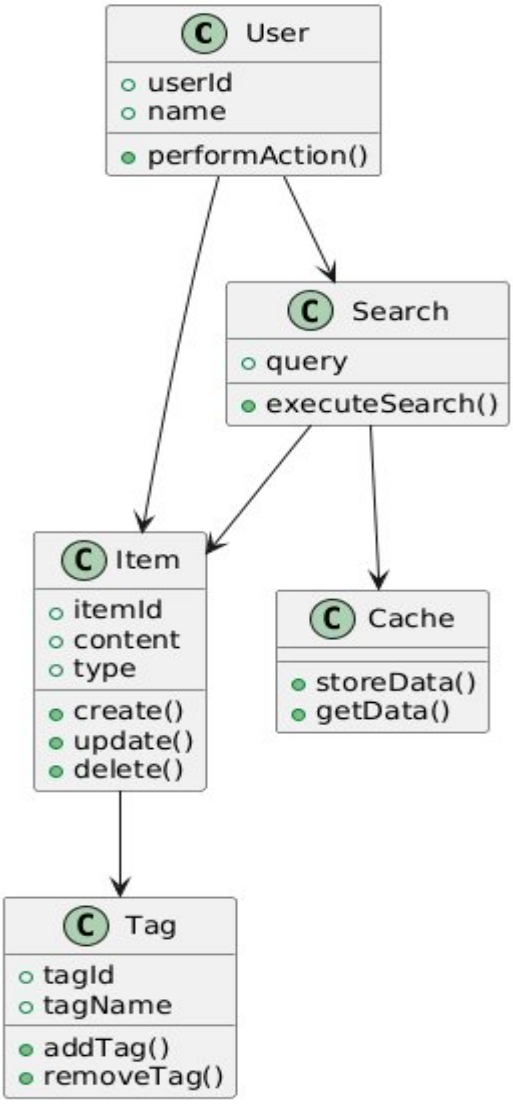


Fig : 4.3.3 Class diagram

The Class Diagram provides a detailed representation of the internal structure of the system. It defines the main classes such as User, Item, Tag, Search, and Cache, along with their attributes and methods. Each class represents a key entity within the system, and its attributes define the data it holds, while its methods represent the operations it can perform. The relationships between classes, such as associations and dependencies, are also illustrated to show how different components are connected.

This diagram is important for understanding how the system is organized at a structural level. It helps in designing the database schema and backend logic by clearly defining how data is stored and accessed. Additionally, it ensures that the system follows a structured

and object-oriented approach, making it easier to maintain, modify, and extend in the future.

4.3.4 ACTIVITY DIAGRAM:

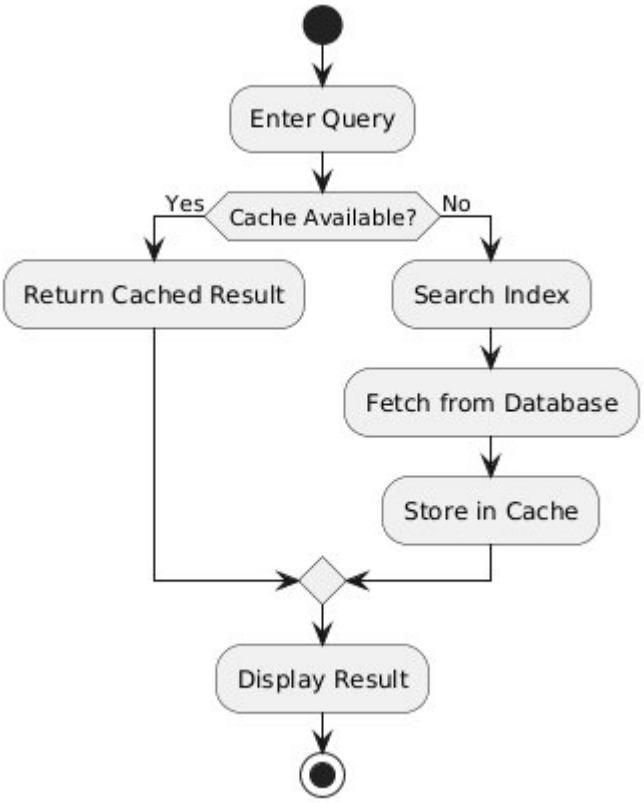


Fig : 4.3.4 Activity diagram

The Activity Diagram represents the workflow of the system by illustrating the sequence of actions involved in processing a task. It begins with the user input and follows the flow through various steps such as checking the cache, retrieving data from the database, and displaying results. Decision points are included in the diagram to represent conditions such as whether the requested data is available in cache or not.

This diagram provides a clear and logical representation of how the system operates, making it easier to understand the flow of control. It is useful for analyzing system behavior and identifying possible improvements in the process. By representing both normal and alternative flows, the Activity Diagram ensures that all possible scenarios are considered during system design.

4.3.5 COLLABORATION DIAGRAM:

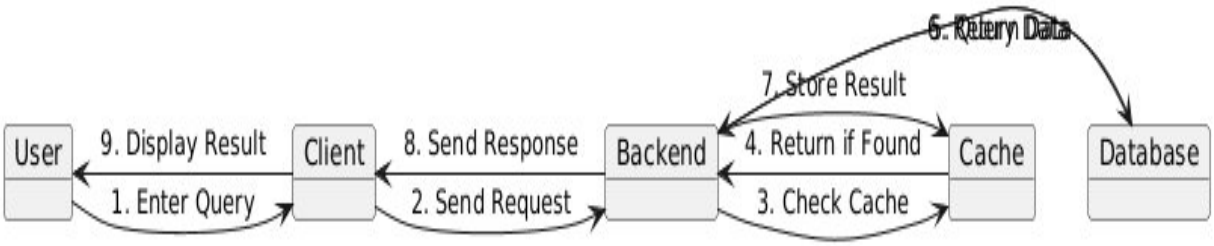


Fig : 4.3.5 Collaboration diagram

The Collaboration Diagram focuses on how different components of the system communicate and work together to perform a task. It shows the interaction between objects such as the client, backend, cache, and database through message passing. Each interaction is represented with numbered messages to indicate the sequence of communication.

This diagram emphasizes the relationships between components and how they collaborate to achieve system functionality. It provides a clear understanding of how responsibilities are distributed among different parts of the system. By visualizing communication patterns, the Collaboration Diagram helps in designing efficient interaction between system components.

4.3.6 STATE CHART DIAGRAM:

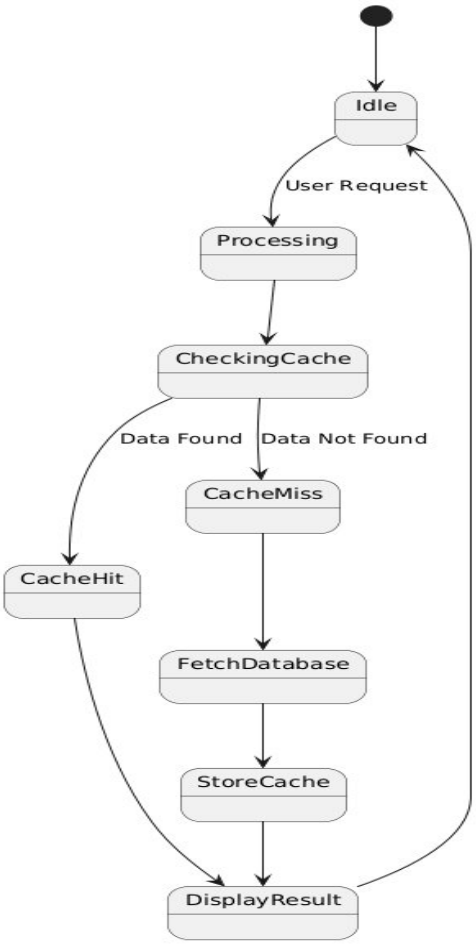


Fig : 4.3.6 State chart diagram

The Statechart Diagram describes the different states of the system and the transitions between them during execution. It shows how the system moves from one state to another based on user actions or internal conditions. Typical states include idle, processing request, checking cache, fetching data from the database, and displaying results. Each transition is triggered by a specific event or condition.

This diagram is useful for understanding how the system behaves dynamically over time. It helps in modeling system behavior in response to different inputs and ensures that all possible states are accounted for. By clearly defining state transitions, the diagram improves system reliability and helps prevent unexpected behavior during execution.

4.3.7 COMPONENT DIAGRAM:

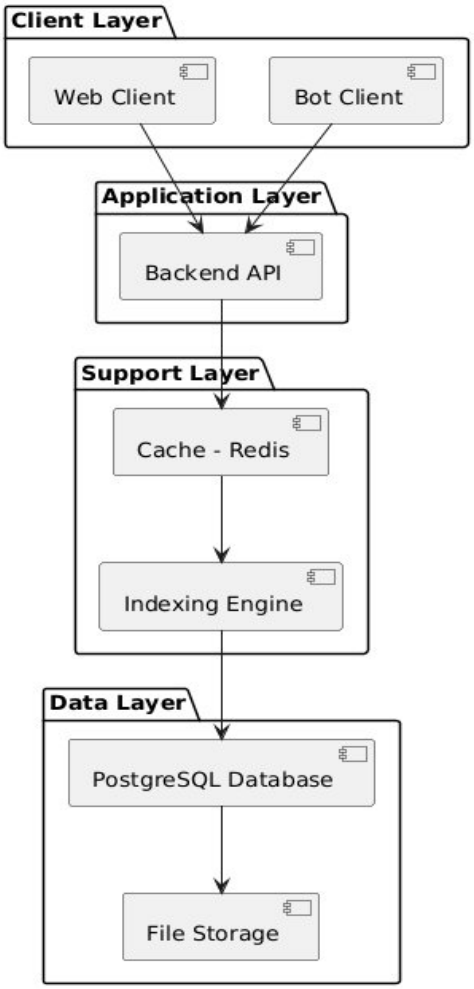


Fig : 4.3.7 Component diagram

The Component Diagram represents the system at a higher level by dividing it into independent modules or components. These components include the client layer, backend API, caching system, indexing module, and database. Each component performs a specific function and interacts with other components through defined interfaces.

This diagram highlights the modular structure of the system, which is essential for scalability and maintainability. By separating the system into smaller components, it becomes easier to update, test, and extend individual parts without affecting the entire system. The Component Diagram provides a clear overview of how the system is organized and how different modules are interconnected.

4.3.8 DEPLOYMENT DIAGRAM:

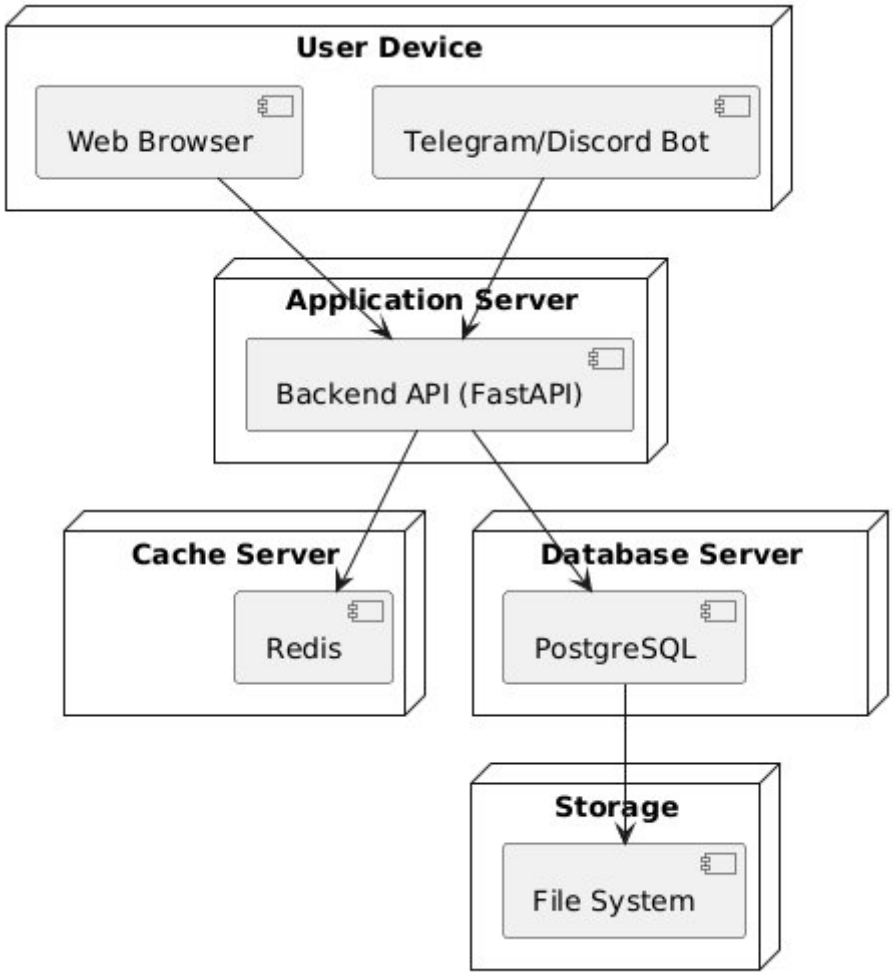


Fig : 4.3.8 Deployment diagram

The Deployment Diagram illustrates the physical implementation of the system by showing how components are distributed across hardware nodes. It represents devices such as user systems, application servers, database servers, and storage systems. The diagram also shows how these nodes are connected and how data flows between them.

This diagram is important for understanding how the system is deployed in a real-world environment. It helps in planning system infrastructure and ensures that all components are properly connected. By visualizing the physical arrangement of the system, the Deployment Diagram provides insights into performance, scalability, and resource utilization.

5.METHODOLOGY

The development of the **Indexed Knowledge Storage and Retrieval System** follows a structured approach that includes system design, modular implementation, and performance optimization. The methodology focuses on organizing data using tag-based indexing and enabling fast retrieval through efficient backend processing. The system is built using a client–server architecture and follows an incremental development model to ensure reliability and maintainability.

5.1 MODULE DESCRIPTION

The system is divided into the following modules:

1. Data Management Module

- Handles storage of knowledge items such as notes, images, and links
- Performs CRUD operations (Create, Read, Update, Delete)
- Maintains file references for stored media

2. Tag Management Module

- Allows assigning multiple tags to each item
- Maintains mapping between tags and items
- Supports updating and deleting tags

3. Indexing Module

- Creates and maintains tag-based indexes
- Enables direct lookup of items without scanning entire data
- Updates index dynamically when data changes

4. Search and Retrieval Module

- Processes user queries (simple tag or structured query)
- Retrieves relevant data using indexing
- Ensures fast and efficient response

5. Caching Module

- Stores frequently accessed data using Redis
- Reduces database load and improves performance
- Handles cache updates and invalidation

6. API Module

- Developed using FastAPI
- Handles communication between client and server
- Exposes RESTful endpoints for all operations

5.2 PROCESS / ALGORITHM

The system follows a step-by-step process for storing and retrieving data:

A. Data Storage Process

1. User sends data (note/image/link) through client interface
2. Backend receives and validates the request
3. Data is stored in PostgreSQL
4. Tags are extracted and stored
5. Index is updated with tag-to-item mapping
6. Confirmation is sent to the user

B. Data Retrieval Process

1. User enters a tag or query
2. Backend receives the request
3. System checks cache for existing result
    - If found → return cached data
    - If not found → proceed to database
4. Index is used to locate matching items
5. Data is retrieved from database
6. Results are returned to user
7. Frequently accessed results are stored in cache

C. Algorithm (Simplified)

```
Input: Query (tags)
Output: Matching knowledge items

1. Receive query
2. Check cache
3. If result exists:
       return cached result
4. Else:
       search index for matching item IDs
       fetch items from database
       store result in cache
       return result
```

6.IMPLEMENTATION

6.1 SAMPLE CODE

Main.py:

```
from fastapi import FastAPI, File, UploadFile
from fastapi.middleware.cors import CORSMiddleware
import os
from ContentItem import ContentCreate, ImageCreate
from pool import connection_pool
from table_edit import insert_data_into_db, fetch_data , delete_data
from cloudinary import uploader
import cloudinary_config
from apscheduler.schedulers.background import BackgroundScheduler
from apscheduler.triggers.interval import IntervalTrigger

app = FastAPI()

# Configure CORS
app.add_middleware(
    CORSMiddleware,
    allow_origins=["http://localhost:5173", "http://127.0.0.1:5173", "http://localhost:3000",
"http://127.0.0.1:3000", "*"],
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

@app.get("/all")
async def app_get_root():
    conn = connection_pool.getconn()
    try:
        with conn.cursor() as cur:
            cur.execute("SELECT lookup_id FROM knowledge_base;")
```

```
            rows = cur.fetchall()
            return {"data": rows}
    finally:
        connection_pool.putconn(conn)

lis = []

@app.post("/data")
async def app_post_Data(data:ContentCreate):
    insert_data_into_db(data)

@app.post("/upload-image")
async def upload_image(file: UploadFile = File(...)):

    # Validate
    if not file.content_type or not file.content_type.startswith("image/"):
        return {"error": "Only image files allowed"}

    result = uploader.upload(
        file.file,
        folder="knowledge_base_images",
    )

    conn = connection_pool.getconn()
    try:
        with conn.cursor() as cur:
            cur.execute(
                """
                INSERT INTO images (knowledge_base_id, public_id, image_name, position)
                VALUES (NULL, %s, %s, 0)
                RETURNING id;
                """,
                (result["public_id"], file.filename)
            )
```

```
            image_id = cur.fetchone()[0]
        conn.commit()
    finally:
        connection_pool.putconn(conn)

    return {
        "image_id": image_id
    }

@app.get("/data/{lookup_id}")
async def get_data(lookup_id: str):
    data = fetch_data(lookup_id)
    if data:
        return data
    else:
        return {"error": "Data not found"}

@app.delete("/data/{lookup_id}")
async def delete_data_route(lookup_id: str):
    existing_data = fetch_data(lookup_id)
    if not existing_data:
        return {"error": "Data not found"}
    delete_data(lookup_id)
    return {"message": "Data deleted successfully"}

def cleanup_detached_images():
    """Delete all detached images from the database and cloudinary."""
    conn = connection_pool.getconn()
    try:
        with conn.cursor() as cur:
            # Find images not associated with any knowledge base item
            cur.execute("""
                SELECT id, public_id FROM images
                WHERE knowledge_base_id IS NULL;
```

```
            """)
            detached_images = cur.fetchall()

            for image_id, public_id in detached_images:
                # Delete from cloudinary
                uploader.destroy(public_id)

                # Delete from database
                cur.execute("DELETE FROM images WHERE id = %s", (image_id,))

        conn.commit()
    finally:
        connection_pool.putconn(conn)

# Initialize scheduler
scheduler = BackgroundScheduler()
scheduler.add_job(cleanup_detached_images, 'interval', minutes=1)
scheduler.start()

@app.on_event("shutdown")
def shutdown_event():
    scheduler.shutdown()


if __name__ == '__main__':
    os.system('fastapi dev main.py')
```

App. jsx

```
 import { useState, useRef, useEffect } from 'react'
import axios from 'axios'
import './App.css'
import ContentCreation from './ContentCreation'
import ContentDisplay from './ContentDisplay'
import LookupInput from './LookupInput'

function App() {
  const [lookupId, setLookupId] = useState('')
  const [data, setData] = useState(null)
  const [showCreate, setShowCreate] = useState(false)
  const [searched, setSearched] = useState(false)
  const [showLookupModal, setShowLookupModal] = useState(false)
  const lookupRef = useRef(null)

  const getData = async (id) => {
    const trimmed = (id || '').trim()
    if (!trimmed) return
    setSearched(true)
    try {
      const resp = await axios.get(`http://127.0.0.1:8000/data/${trimmed}`)
      setData(resp.data)
    } catch (err) {
      console.error('Error fetching data:', err)
      setData(null)
    }
  }

  const handleKeyDown = (e) => {
    if (e.key === 'Enter') getData(lookupId)
  }
```

```
// Global shortcut: Ctrl+K or Meta+K to open the lookup modal
useEffect(() => {
  const handler = (e) => {
    const key = e.key?.toLowerCase()
    if ((e.ctrlKey || e.metaKey) && key === 'k') {
      e.preventDefault()
      setShowLookupModal(true)
      // focus after render
      setTimeout(() => lookupRef.current?.focus(), 50)
    }
    if (e.key === 'Escape' && showLookupModal) {
      setShowLookupModal(false)
    }
  }
  window.addEventListener('keydown', handler)
  return () => window.removeEventListener('keydown', handler)
}, [showLookupModal])

const resetToHome = () => {
  setData(null)
  setLookupId('')
  setSearched(false)
}

// If an item is found we show the ContentDisplay view.
if (data) {
  return (
    <div>
      <header className="fixed top-0 left-0 right-0 z-50 bg-[#0e0e0e]/80 backdrop-blur-
xl flex justify-between items-center px-8 h-16">
        <div className="text-lg font-bold tracking-tighter text-primary font-headline
uppercase cursor-pointer" onClick={resetToHome}>KnowledgeBASE</div>
        <div className="flex items-center gap-4">
```

```
          <button    className="p-2    rounded-full    hover:bg-[#252626]    text-[#acabaa]"
onClick={() => setShowCreate(true)}>
            <span className="material-symbols-outlined">add</span>
          </button>
        </div>
      </header>

      <main className="pt-24 px-8">
        <ContentDisplay data={data} />
      </main>

      {showCreate && (
        <div className="fixed inset-0 bg-black/60 z-60 flex items-center justify-center">
          <div className="bg-surface-container-high p-6 rounded-lg w-full max-w-3xl">
            <div className="flex justify-between items-center mb-4">
              <h2 className="text-xl">Create Content Item</h2>
              <button onClick={() => setShowCreate(false)} className="material-symbols-
outlined">close</button>
            </div>
            <ContentCreation />
          </div>
        </div>
      )}
      {showLookupModal && (
        <div className="fixed inset-0 bg-black/60 z-50 flex items-center justify-center">
          <div className="w-full max-w-xl px-6">
            <div className="bg-surface-container-high p-6 rounded-lg">
              <div className="flex items-center gap-3 mb-4">
                <LookupInput
                  ref={lookupRef}
                  value={lookupId}
                  onChange={(e) => setLookupId(e.target.value)}
                  onSearch={() => { getData(lookupId); setShowLookupModal(false) }}
                  onClose={() => setShowLookupModal(false)}
```

```
                />
              </div>
              <div className="flex justify-end gap-2">
                <button className="px-4 py-2 rounded border" onClick={() =>
setShowLookupModal(false)}>Cancel</button>
              <button className="px-4 py-2 rounded bg-primary text-black" onClick={() =>
{ getData(lookupId); setShowLookupModal(false) }}>Go</button>
              </div>
            </div>
          </div>
        </div>
      )}
    </div>
  )
}

// Landing / search view
return (
  <div className="min-h-screen bg-background text-on-surface flex flex-col">
    <header className="fixed top-0 left-0 right-0 z-50 bg-[#0e0e0e]/80 backdrop-blur-xl
flex justify-between items-center px-8 h-16">
      <div className="text-lg font-bold tracking-tighter text-primary font-headline
uppercase">KnowledgeBASE</div>
      <div className="flex items-center gap-4">
        <button className="px-4 py-2 rounded bg-primary text-black font-semibold"
onClick={() => setShowCreate(true)}>Create Content Item</button>
      </div>
    </header>

    <div className="flex-1 flex items-center justify-center px-4">
      <div className="w-full max-w-2xl">
        <div className="flex flex-col items-center gap-6 py-24">
          <LookupInput
            value={lookupId}
```

```
            onChange={(e) => setLookupId(e.target.value)}
            onSearch={() => getData(lookupId)}
          />
          <div className="flex gap-4">
            <button className="px-6 py-3 rounded bg-primary text-black font-semibold"
onClick={() => getData(lookupId)}>Search</button>
            <button className="px-6 py-3 rounded border border-outline-variant text-on-
surface-variant" onClick={() => { setLookupId(''); setSearched(false); }}>Clear</button>
          </div>
          {searched && !data && <div className="text-error mt-4">Lookup ID does not
exist</div>}
        </div>
      </div>
    </div>

    {showCreate && (
      <div className="fixed inset-0 bg-black/60 z-60 flex items-center justify-center">
        <div className="bg-surface-container-high p-6 rounded-lg w-full max-w-3xl">
          <div className="flex justify-between items-center mb-4">
            <h2 className="text-xl">Create Content Item</h2>
            <button onClick={() => setShowCreate(false)} className="material-symbols-
outlined">close</button>
          </div>
          <ContentCreation />
        </div>
      </div>
    )}
  </div>
 )
}

export default App
```

ContentDisplay.jsx

```
import Markdown from "./Markdown"
import ImageCard from "./ImageCard"

function ContentDisplay({ data }) {
  const title = data?.title || data?.name || "Untitled";
  const date = data?.date || data?.created_at || "—";
  const authors = data?.authors || (data?.author ? [data.author] : null) || data?.group_name ||
[];
  const images = data?.images || [];

  if (!data) return <p className="text-on-surface">No data</p>

  return (
    <div className="bg-background font-body text-on-surface selection:bg-primary/30">
      <main className="pt-24 pb-32 max-w-[1400px] mx-auto px-8 lg:px-16 flex flex-col
gap-24">
        <div className="flex flex-col lg:flex-row gap-16 items-start">
          <section className="lg:w-[70%] prose max-w-none">
            <header className="mb-16">
              <span className="font-label text-[10px] uppercase tracking-widest text-primary
mb-4 block">{data?.category || 'Research Memo / Knowledge'}</span>
              <h1 className="font-headline">{title}</h1>
              <div className="flex items-center gap-6 text-on-surface-variant text-sm font-
label uppercase tracking-wider mt-8">
                <span className="flex items-center gap-2"><span className="material-
symbols-outlined text-xs">calendar_today</span> {date}</span>
                {authors && authors.length > 0 && <span className="flex items-center gap-
2"><span className="material-symbols-outlined text-xs">person</span>
{Array.isArray(authors) ? authors.join(', ') : authors}</span>}
              </div>
```

```
          </header>

          {data.md_notes ? (
            <Markdown content={data.md_notes} />
          ) : (
            <p className="text-on-surface-variant">No article body available.</p>
          )}
        </section>

        <aside className="lg:w-[30%] lg:sticky lg:top-24 max-h-[calc(100vh-160px)] overflow-y-auto pr-2 flex flex-col gap-6">
          {images.length === 0 ? (
            <div className="p-4 bg-surface-container-low rounded-xl text-on-surface-variant">No images</div>
          ) : (
            <div className="grid grid-cols-1 sm:grid-cols-2 gap-4">
              {images.map((img, idx) => (
                <ImageCard key={idx} img={img} idx={idx} />
              ))}
            </div>
          )}
        </aside>
      </div>

      <section className="w-full border-t border-outline-variant/5 pt-16">
        <div className="grid md:grid-cols-2 gap-16">
          <div className="space-y-8">
            <div className="flex items-center gap-3">
              <span className="material-symbols-outlined text-primary">hub</span>
            <h3 className="font-label text-xs font-bold uppercase tracking-[0.2em] text-on-surface">External Sources</h3>
            </div>
            <div className="grid gap-4">
              {(data.article_links || []).map((link, i) => (
```

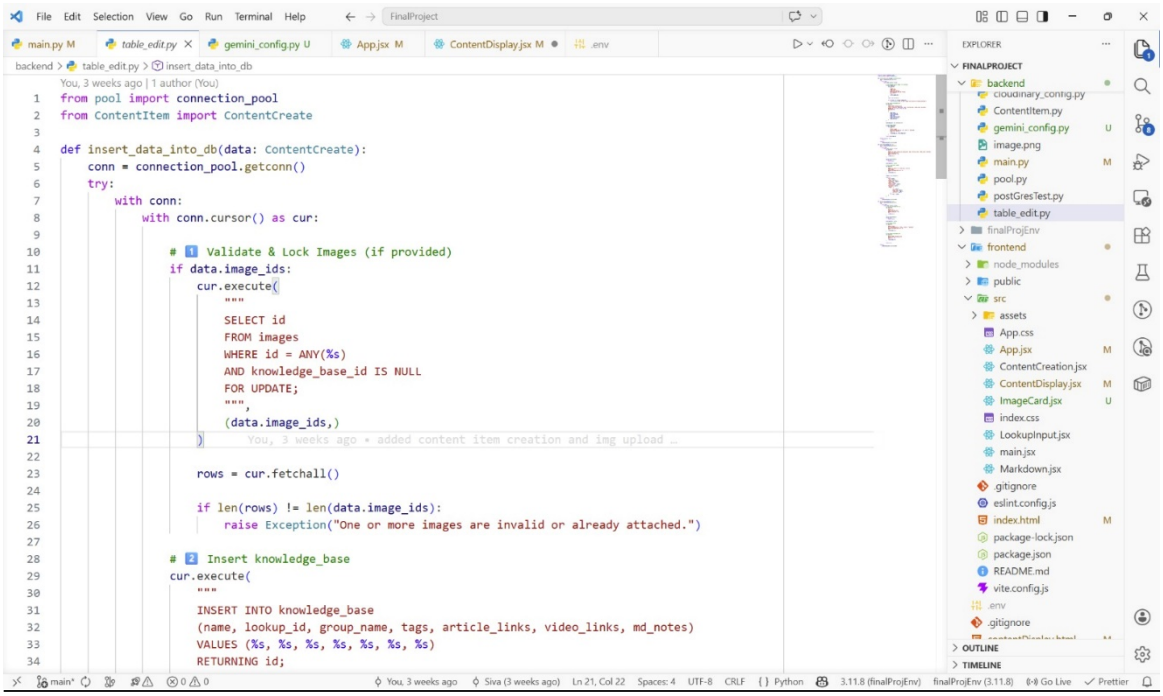
```
<a
  key={i}
 className="flex items-center gap-4 p-4 rounded-lg bg-surface-container-low hover:bg-surface-container-high transition-all group"
  href={typeof link === 'string' ? link : (link.url || '#')}
  target="_blank"
  rel="noopener noreferrer"
 >
<div className="w-10 h-10 rounded flex items-center justify-center bg-primary-container/20 text-primary">
  <span className="material-symbols-outlined">article</span>
 </div>
 <div className="flex-1">
  <div className="text-sm font-medium text-on-surface group-hover:text-primary transition-colors">{link.title || link}</div>
  <div className="text-[10px] text-on-surface-variant font-label uppercase tracking-wider">{link.source || ''}</div>
 </div>
 <span className="material-symbols-outlined text-on-surface-variant text-sm">open_in_new</span>
</a>
))}

{(data.video_links || []).length > 0 && (
 <div className="mt-4">
  <h4 className="font-label text-xs uppercase tracking-wider text-on-surface-variant">Video Links</h4>
  <div className="grid gap-4 mt-2">
   {(data.video_links || []).map((link, idx) => (
 <a key={idx} className="flex items-center gap-4 p-4 rounded-lg bg-surface-container-low hover:bg-surface-container-high transition-all group" href={typeof link === 'string' ? link : (link.url || '#')} target="_blank" rel="noopener noreferrer">
     <div className="w-10 h-10 rounded flex items-center justify-center bg-primary-container/20 text-primary">
```

```
                <span className="material-symbols-outlined">play_circle</span>
              </div>
              <div className="flex-1">
            <div className="text-sm font-medium text-on-surface group-hover:text-primary transition-colors">{(typeof link === 'string') ? link : (link.title || link)}</div>
            <div className="text-[10px] text-on-surface-variant font-label uppercase tracking-wider">{(typeof link === 'string') ? '' : (link.source || '')}</div>
              </div>
            <span className="material-symbols-outlined text-on-surface-variant text-sm">open_in_new</span>
              </a>
            ))}

          </div>
        </div>
      )}
    </div>
  </div>


  </div>
 </section>
</main>

<footer className="bg-[#000000] w-full py-12 px-8">
 <div className="bg-[#191a1a] h-px w-full mb-8"></div>
 <div className="max-w-7xl mx-auto flex flex-col md:flex-row justify-between items-center gap-6">
  <div className="font-['Manrope'] uppercase tracking-widest text-[10px] text-[#acabaa]">© 2024 Obsidian Flow Knowledge Management</div>
  <div className="flex gap-8">
   <a className="font-['Manrope'] uppercase tracking-widest text-[10px] text-[#acabaa] hover:text-[#e7e5e5] transition-colors opacity-80 hover:opacity-100" href="#">Documentation</a>
```

```
          <a className="font-['Manrope'] uppercase tracking-widest text-[10px] text-[#acabaa] hover:text-[#e7e5e5] transition-colors opacity-80 hover:opacity-100" href="#">Privacy</a>
          <a className="font-['Manrope'] uppercase tracking-widest text-[10px] text-[#acabaa] hover:text-[#e7e5e5] transition-colors opacity-80 hover:opacity-100" href="#">Support</a>
        </div>
      </div>
    </footer>
  </div>
 )
}

export default ContentDisplay
```

6.2 OUTPUT SCREENS:

6.2.1 Code snippet in IDE



```python
from pool import connection_pool
from ContentItem import ContentCreate

def insert_data_into_db(data: ContentCreate):
    conn = connection_pool.getconn()
    try:
        with conn:
            with conn.cursor() as cur:

                # 1 Validate & Lock Images (if provided)
                if data.image_ids:
                    cur.execute(
                        """
                        SELECT id
                        FROM images
                        WHERE id = ANY(%s)
                        AND knowledge_base_id IS NULL
                        FOR UPDATE;
                        """,
                        (data.image_ids,)
                    )

                    rows = cur.fetchall()

                    if len(rows) != len(data.image_ids):
                        raise Exception("One or more images are invalid or already attached.")

                # 2 Insert knowledge_base
                cur.execute(
                    """
                    INSERT INTO knowledge_base
                    (name, lookup_id, group_name, tags, article_links, video_links, md_notes)
                    VALUES (%s, %s, %s, %s, %s, %s, %s)
                    RETURNING id;
```

Fig : 6.2.1 Code snippet in IDE

This screen shows the backend implementation of the system in a development environment. It highlights the server-side logic responsible for inserting and managing knowledge items in the database. The code includes database connection handling, validation of input data, and execution of SQL queries to store structured information such as tags, links, and notes. It ensures that data integrity is maintained by validating inputs before insertion.

The backend is structured to handle multiple operations efficiently, including checking constraints and managing relationships between different data elements. This layer plays a critical role in ensuring that the system functions correctly by processing user inputs and storing them in the database. It reflects the technical foundation of the system and demonstrates how data flows from the user interface to persistent storage.

6.2.2 MAIN LANDING PAGE

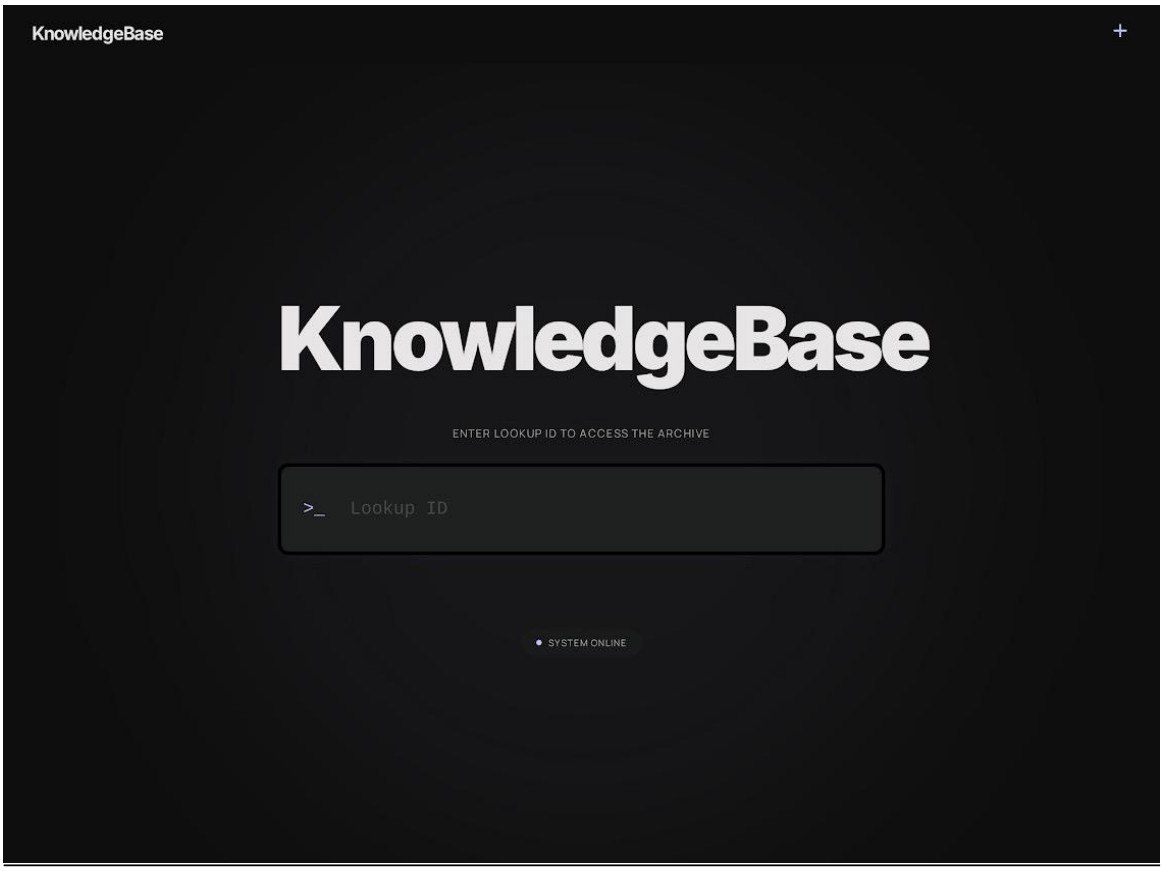


Fig : 6.2.2 Main Landing page

This screen represents the entry point of the system, where users can access stored knowledge using a unique lookup ID. The interface is simple and focused, with a clean design that minimizes distractions. Users are prompted to enter a lookup ID, which acts as a key to retrieve specific knowledge entries from the database. The system status indicator confirms that the system is active and ready for input.

The design ensures ease of use by providing a straightforward input mechanism and clear instructions. This screen is important for controlling access to stored data and enabling quick retrieval. It acts as a gateway between the user and the knowledge base, ensuring that the interaction starts in a structured and controlled manner.

6.2.3 CONTENT DISPLAY PAGE

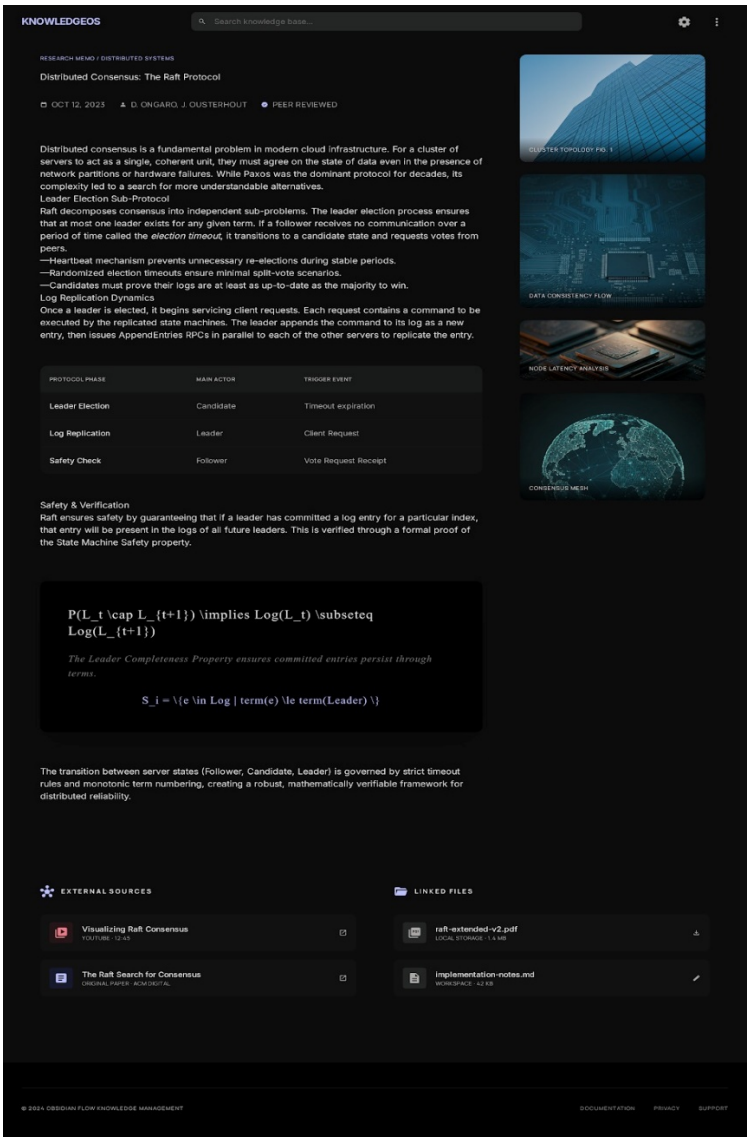


Fig : 6.2.3 Content Display Page

This screen displays detailed information about a selected knowledge item. It includes content such as text descriptions, images, diagrams, and additional references. The layout is structured to present information clearly, with headings, sections, and supporting visuals. It allows users to understand complex topics through organized content presentation.

Additional features such as linked files, external resources, and categorized sections enhance the usefulness of this interface. Users can explore related content and access supporting materials easily. This screen demonstrates how the system transforms stored data into meaningful and accessible knowledge for the user..

6.2.4 CONTENT CREATION PAGE

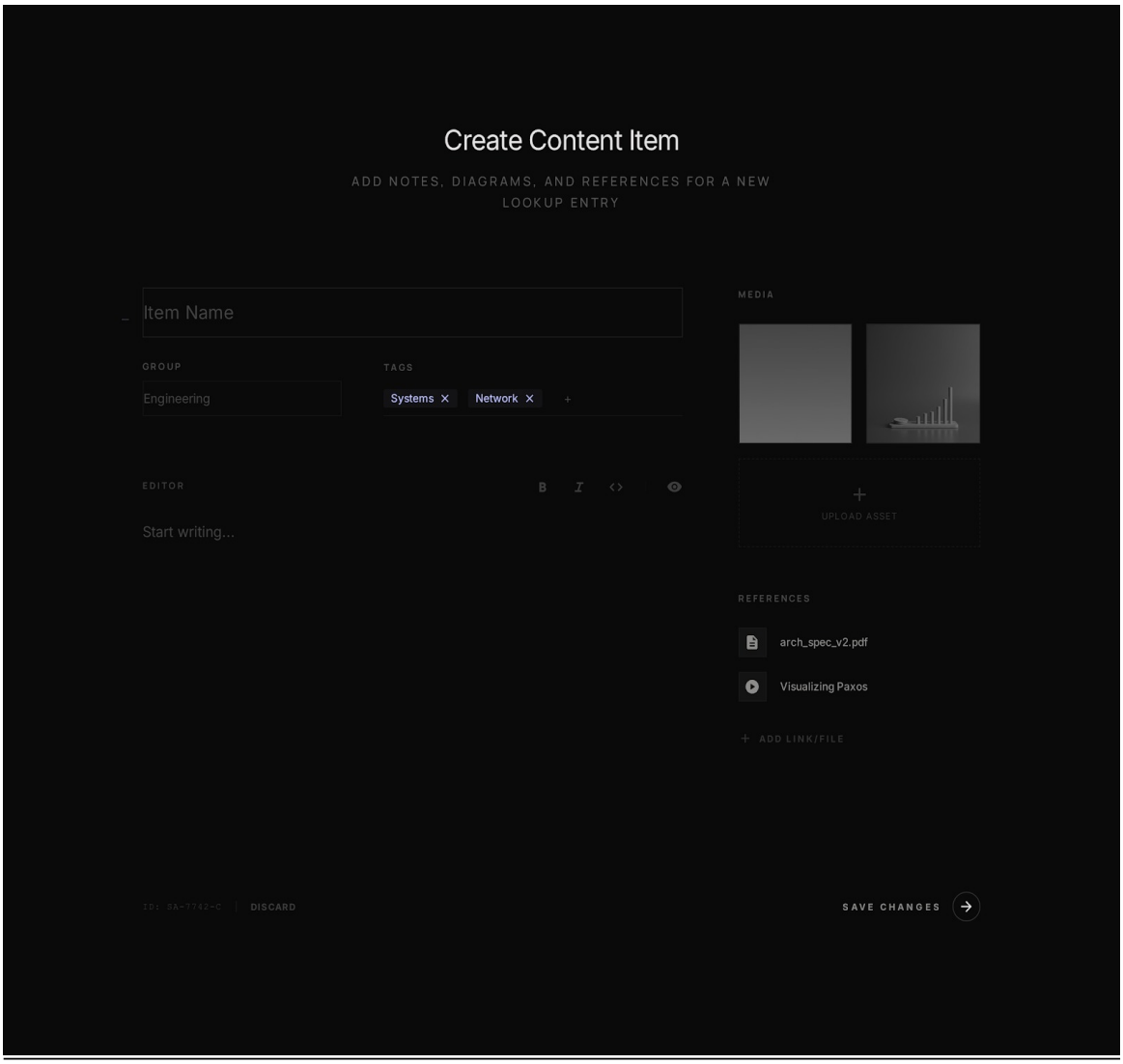


Fig 6.2.4 : Content creation page

This screen allows users to create and manage new knowledge entries within the system. It includes fields for entering item details such as name, group, tags, and textual content. Users can also upload media files and attach references such as documents or links. The interface is designed to be user-friendly, allowing efficient input of structured data.

The form-based layout ensures that all necessary information is captured before storing the data. Tagging functionality allows flexible organization, enabling better indexing and retrieval later. This screen plays a key role in maintaining the quality and structure of the knowledge base by ensuring that data is entered in a consistent and organized manner.

6.2.5 RAG chatbot interface

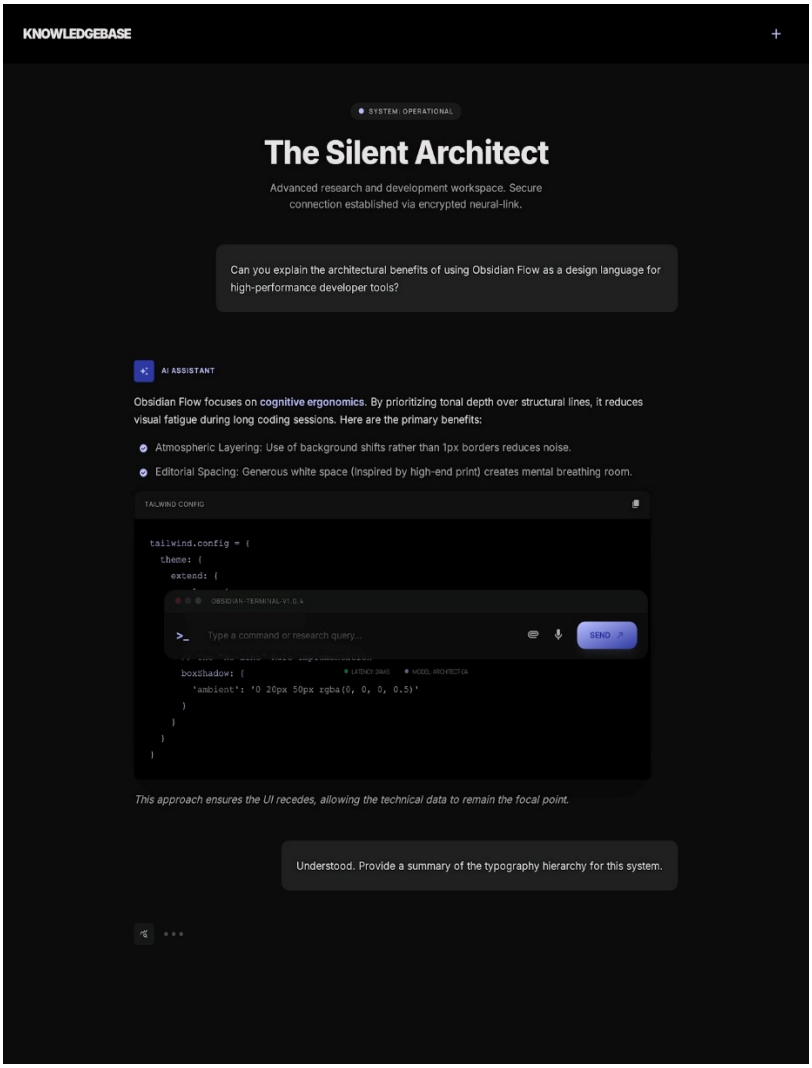


Fig 6.2.5 : RAG chatbot interface

This screen represents the advanced interaction interface of the system, where the user can engage with the knowledge base through a conversational or query-driven approach. The interface is designed with a minimal and dark-themed layout to reduce visual fatigue and improve focus. It displays system status indicators such as “System Operational,” ensuring the user that the system is active. The central section allows users to input queries or requests related to stored knowledge, making it easier to retrieve or analyze information.

The response area shows processed outputs generated by the system, including structured explanations and formatted content. The design emphasizes readability through proper spacing, typography, and layering. This interface acts as a bridge between raw data storage and intelligent knowledge retrieval, enhancing usability by allowing users to interact with the system in a more intuitive and natural way.

7.CONCLUSION & FUTURE SCOPE

Future Scope

Although the current system meets its core objectives, several enhancements can be implemented in the future to extend its functionality and usability:

- Implementation of cloud synchronization for remote access across multiple devices
- Development of a mobile application for improved accessibility
- Enhancement of query capabilities with advanced filtering and search options
- Integration of user authentication and multi-user support
- Addition of analytics to track usage patterns and improve data organization
- Support for automatic backups and recovery mechanisms
- Integration with external platforms and tools for extended functionality

The future improvements aim to enhance scalability, usability, and feature richness while maintaining the system's core principles of simplicity and efficiency.

Conclusion:

The **Indexed Knowledge Storage and Retrieval System** successfully addresses the challenges of managing and retrieving personal knowledge efficiently. By replacing traditional file-based organization and full-text search methods with a structured tag-based indexing approach, the system significantly reduces retrieval time and improves overall performance. The integration of indexing and caching ensures that frequently accessed data can be retrieved quickly with minimal latency.

The system supports multiple content types such as notes, images, and links, providing flexibility in data management. Its modular architecture and use of reliable technologies like PostgreSQL, FastAPI, and Redis make it efficient, scalable, and easy to maintain. The local-first design ensures data ownership, reliability, and independence from cloud-based services. Overall, the project demonstrates a practical and effective solution for personal knowledge management with strong system design principles.

.

8.BIBLIOGRAPHY

The following references were used in the study and development of the Indexed Knowledge Storage and Retrieval System:

1. PostgreSQL Documentation, *PostgreSQL Global Development Group*, Available: https://www.postgresql.org/docs/
2. FastAPI Documentation, *Sebastián Ramírez*, Available: https://fastapi.tiangolo.com/
3. Redis Documentation, *Redis Labs*, Available: https://redis.io/documentation
4. Docker Documentation, *Docker Inc.*, Available: https://docs.docker.com/
5. Notion Official Website, Available: https://www.notion.so/
6. Evernote Official Website, Available: https://evernote.com/
7. Obsidian Official Website, Available: https://obsidian.md/
8. Manning, C. D., Raghavan, P., & Schütze, H., *Introduction to Information Retrieval*, Cambridge University Press, 2008
9. Silberschatz, A., Korth, H. F., & Sudarshan, S., *Database System Concepts*, McGraw-Hill Education
10. Elmasri, R., & Navathe, S. B., *Fundamentals of Database Systems*, Pearson